Université Paris Cité

Licence Informatique – UE Intelligence Artificielle

IA et Onitama

Exploration des algorithmes Minimax et de l'élagage α-β

Simeon TAKENNE MEKEM

Informatique et Application &

Mathématiques et Applications

Amine ZEROUALI

Informatique et Application &

Mathématiques et Applications

Responsable de l'UE : Elise Bonzon

Code source du projet

Résumé : Ce document présente la conception d'un moteur de jeu pour *Onitama* ainsi que l'implémentation de trois niveaux de difficulté basés sur l'algorithme Minimax. L'efficacité de l'élagage α-β est démontrée par une analyse du nombre de nœuds explorés et une étude statistique portant sur 1000 parties par configuration. Les résultats confirment la supériorité des heuristiques positionnelles sur le simple comptage matériel.

Février – Mai 2026

Table des matières

1 Introduction **2**

2 Présentation du jeu **2**

2.1 Description générale 2

2.2 Règles du jeu 2

2.3 Intérêt stratégique et justification du choix 3

3 Implémentation du moteur de jeu **3**

3.1 Représentation de l'état 3

3.2 Génération des coups 4

3.3 Application et annulation des coups 5

3.4 Détection des états terminaux 5

3.5 Tests unitaires 5

4 Intelligence artificielle **6**

4.1 Algorithme Minimax 6

4.2 Élagage α-β 7

4.3 Fonctions d'évaluation (heuristiques) 8

4.4 Profils des IA et niveaux de difficulté 9

5 Analyse expérimentale **10**

5.1 Temps de calcul par profondeur 10

5.2 Nombre de nœuds explorés 11

5.3 Impact de l'élagage α-β sur l'efficacité 12

6 Tournoi entre les intelligences artificielles **13**

6.1 Protocole expérimental 13

6.2 Résultats 13

6.3 Évolution du taux de victoire 13

6.4 Analyse des résultats 14

7 Utilisation de l'IA générative **15**

8 Difficultés rencontrées **16**

9 Conclusion **18**

A Code source principal **20**

A.1 Évaluation heuristique H2 - Matériel dynamique 20

A.2 Script de génération des graphiques 20

B Dictionnaire des 16 cartes **21**

1 Introduction

Ce rapport présente notre projet d'intelligence artificielle appliqué au jeu de société *Onitama*. L'objectif est de concevoir un moteur de jeu complet ainsi que plusieurs agents artificiels de niveaux de difficulté croissants, en s'appuyant sur les algorithmes classiques de la théorie des jeux.

Dans ce rapport, nous décrirons dans un premier temps les règles du jeu et les raisons de notre choix (Section 2), nous présenterons ensuite les détails d'implémentation du moteur de jeu (Section 3), puis les algorithmes d'IA et nos trois heuristiques (Section 4). Nous analyserons enfin les résultats de nos tournois (Section 6) avant de conclure (Section 9).

2 Présentation du jeu

2.1 Description générale

Onitama est un jeu de stratégie à deux joueurs, déterministe et à information parfaite, créé par Shimpei Sato. Il se joue sur un plateau de 5×5 cases. Chaque joueur dispose de cinq pièces : quatre **élèves** et un **maître**.

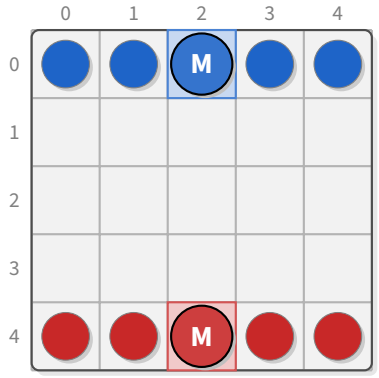

FIGURE 1 – État initial du plateau.
Chaque joueur possède quatre pions et un maître (noté M). Les cases centrales colorées représentent les temples.
Convention `plateau[y][x]` : la ligne $y = 0$ correspond au haut du plateau (Joueur Bleu) et $y = 4$ au bas (Joueur Rouge).

2.2 Règles du jeu

Au début de chaque partie, cinq cartes de mouvement sont tirées aléatoirement parmi seize. Deux cartes sont attribuées à chaque joueur et la dernière, appelée **carte extra**, déterminera quant à elle quel joueur commence :

— si sa couleur est bleue, le Joueur 1 commence

— si elle est rouge, c'est le Joueur 2.

Déroulement d'un tour. Le joueur actif choisit l'une de ses deux cartes et déplace l'une de ses pièces selon les mouvements autorisés par cette carte. Les déplacements sont relatifs à l'orientation du joueur (le Joueur 2 joue dans le sens symétrique). La carte jouée prend la place de la carte extra. Cette dernière est récupérée par le joueur, tandis que la carte qu'il vient de jouer sera intégrée à la main de l'adversaire à la fin de son prochain tour.

Conditions de victoire. Un joueur remporte la partie selon l'un des deux chemins :

— **Voie de la Pierre** : capturer le maître adverse ;

— **Voie du Courant** : amener son propre maître sur le temple adverse (case $(2, 0)$ pour le Joueur 1, case $(2, 4)$ pour le Joueur 2).

2.3 Intérêt stratégique et justification du choix

Onitama présente plusieurs propriétés qui en font un choix particulièrement adapté au cadre de ce projet :

— **Déterminisme et information parfaite** : toutes les informations sont visibles des deux joueurs, ce qui satisfait les hypothèses de l'algorithme Minimax.

— **Espace d'états suffisamment grand** : la combinaison du plateau 5×5 et du tirage aléatoire des cinq cartes parmi seize génère un nombre de parties très élevé, rendant toute résolution par force brute impossible.

— **Originalité et concision** : les règles sont simples à programmer mais le facteur de branchement reste significatif (pouvant atteindre jusqu'à une quarantaine de coups possibles par tour selon les cartes disponibles et la position des pièces), garantissant une profondeur de recherche intéressante.

— **Double condition de victoire** : la coexistence des deux voies de victoire crée des situations stratégiques riches, propices à l'élaboration d'heuristiques nuancées.

3 Implémentation du moteur de jeu

3.1 Représentation de l'état

L'état du jeu est encapsulé dans la classe `Onitama`. Le plateau est représenté par une matrice Python 5×5 d'entiers signés :

Valeur	Signification
0	Case vide
1	Élève du Joueur 1 (Bleu)
2	Maître du Joueur 1 (Bleu)
−1	Élève du Joueur 2 (Rouge)
−2	Maître du Joueur 2 (Rouge)

Ce codage signé permet de déterminer l'appartenance d'une pièce à un joueur par un simple test du signe : `piece * joueur > 0`.

Les cartes sont stockées dans le dictionnaire global `CARTES`, dont chaque entrée contient la liste des déplacements relatifs (paires $(\Delta x, \Delta y)$) et la couleur associée. Les mains des deux joueurs sont représentées par deux listes de noms de cartes, et la carte extra par un unique identifiant. La variable `tour` (± 1) indique quel joueur doit jouer.

3.2 Génération des coups

La méthode `obtenir_coups_possibles(joueur)` parcourt le plateau pour identifier les pièces du joueur actif, puis, pour chaque pièce et pour chaque carte disponible, calcule les positions cibles en appliquant le facteur directionnel (`sens = ±1`). Un coup est valide si la case cible est dans les bornes du plateau et n'est pas occupée par une pièce alliée.

```
for (x, y) in mouvements:
    nx = j + (x * sens)
    ny = i + (y * sens)
    if 0 <= nx < 5 and 0 <= ny < 5:
        cible = self.plateau[ny][nx]
        if cible * joueur <= 0:    # case vide ou piece adverse
            coups.append({"carte": carte, "de": (j, i), "vers":
                (nx, ny)})
```

Listing 1 – Extrait de la génération de coups légaux.

Note sur les règles officielles : Une légère simplification a été apportée concernant le cas très rare où un joueur se retrouve sans aucun mouvement légal (totalement bloqué par ses propres pièces et les bords du plateau). Selon les règles strictes d'Onitama, le joueur doit alors passer son tour mais défausser une carte pour l'échanger avec la carte extra. Dans notre implémentation, pour maintenir l'efficacité de l'arbre de recherche, le tour passe simplement à l'adversaire sans rotation des cartes. Ce cas limite étant exceptionnellement rare en pratique, son impact sur l'évaluation globale des heuristiques est considéré comme négligeable.

3.3 Application et annulation des coups

La méthode `jouer_coup` déplace la pièce, effectue l'échange de cartes et inverse le tour courant. Elle retourne la pièce capturée et l'ancienne carte extra afin de permettre l'annulation du coup par `annuler_coup`, qui restaure l'état complet en ordre inverse. Ce mécanisme est indispensable à l'exploration récursive de l'arbre de jeu sans copie coûteuse de l'état.

3.4 Détection des états terminaux

La méthode `verifier_victoire` parcourt le plateau et retourne `1`, `-1` ou `None` selon que le Joueur 1 a gagné, le Joueur 2 a gagné ou que la partie est encore en cours. Les deux conditions de victoire sont vérifiées simultanément :

```
if piece == 2:            # Maitre bleu
    master_p1_vivant = True
    if i == 4 and j == 2:     # Voie du Courant (temple rouge)
        return 1
elif piece == -2:         # Maitre rouge
    master_p2_vivant = True
    if i == 0 and j == 2:     # Voie du Courant (temple bleu)
        return -1
if not master_p1_vivant: return -1    # Voie de la Pierre
if not master_p2_vivant: return  1
```

Listing 2 – Détection des conditions de victoire.

Bien que le moteur n'impose pas de limite intrinsèque, la boucle principale du jeu (lancer_partie) impose une limite de 1000 tours pour éviter les parties théoriquement infinies, déclarant alors la partie nulle.

3.5 Tests unitaires

Afin de garantir la fiabilité du moteur de jeu avant d'y greffer l'algorithme Minimax, plusieurs tests unitaires ont été implémentés à l'aide de la bibliothèque standard `unittest` de Python.

Ces tests ciblent les composants critiques du jeu et s'assurent qu'aucune régression n'est introduite lors de l'optimisation du code. Les points clés validés sont :

— **L'initialisation de l'état** : vérification de la bonne distribution des cinq cartes (deux par joueur, une au centre) et du bon placement des pièces.

— **La détection des états terminaux** : simulation de positions spécifiques (comme le placement forcé d'un maître sur le temple adverse) pour s'assurer que la méthode

`verifier_victoire` retourne le bon gagnant.

— **La réversibilité des coups (Make/Unmake)** : un test crucial vérifie que l'enchaînement des méthodes `jouer_coup` et `annuler_coup` restaure exactement l'état du plateau, les mains des joueurs, la carte extra et le tour courant. C'est cette validation qui garantit que l'arbre de recherche Minimax ne corrompt pas l'état du jeu lors de son parcours.

L'exécution de cette suite de tests valide le socle logique du projet et permet aux heuristiques de s'appuyer sur des états de jeu rigoureusement exacts.

4 Intelligence artificielle

4.1 Algorithme Minimax

L'algorithme Minimax est un algorithme de recherche dans l'arbre de jeu adapté aux jeux à somme nulle à deux joueurs. Il s'appuie sur la récursivité :

— Si l'état est terminal ou si la profondeur maximale est atteinte, retourner la valeur de l'heuristique ;

— Si c'est le tour du joueur MAX (Bleu), retourner la valeur maximale parmi les enfants ;

— Si c'est le tour du joueur MIN (Rouge), retourner la valeur minimale parmi les enfants.

Dans notre implémentation, les deux fonctions récursives `max_value` et `min_value` partagent les paramètres α et β dès leur premier appel depuis `minimax_decision`, ce qui intègre nativement l'élagage α-β (voir Section 4.2).

Algorithme 1 : Minimax (version MAX)

Input : état s, profondeur d, α, β, heuristique h
Output : valeur de l'état

```
if s est terminal then
    return score terminal
end
if d = 0 then
    return h(s)
end
v ← −∞
foreach coup a applicable dans s do
    s' ← jouer(s, a)
    v ← max{v, MinValue(s', d − 1, α, β, h)}
    annuler(s, a)
    if v ≥ β then
        return v // élagage
    end
    α ← max{α, v}
end
return v
```

4.2 Élagage α-β

L'élagage α-β maintient deux bornes globales :

— α : meilleure valeur garantie pour MAX (initialisé à $-\infty$) ;

— β : meilleure valeur garantie pour MIN (initialisé à $+\infty$).

Lorsque $v \geq \beta$ dans un nœud MAX (respectivement $v \leq \alpha$ dans un nœud MIN), les branches restantes sont élagués car elles ne peuvent améliorer le résultat. Dans le meilleur des cas, cet élagage réduit la complexité temporelle de $O(b^m)$ à $O(b^{m/2})$, doublant ainsi la profondeur de recherche atteignable.

Notre implémentation mélange aléatoirement les coups (`random.shuffle`) avant l'exploration, ce qui introduit de la variété dans les parties tout en empêchant un biais systématique lié à l'ordre de génération.

```python
v = float('-inf')
for coup in coups:
    piece_capturee, ancienne_extra = self.jouer_coup(coup)
    v = max(v, self.min_value(profondeur - 1, alpha, beta,
        heuristique))
```

```
    self.annuler_coup(coup, piece_capturee, ancienne_extra)
    if v >= beta:           # Elagage beta
        return v
    alpha = max(alpha, v)
return v
```

Listing 3 – Élagage α-β dans `max_value`.

4.3 Fonctions d'évaluation (heuristiques)

En cas de non-atteinte d'un état terminal dans les limites de profondeur, l'algorithme recourt à une fonction d'évaluation pour estimer la valeur d'une position. Nous avons implémenté trois heuristiques, toutes exprimées du point de vue du Joueur 1 (MAX) :

Identifiant	Nom court	Description
H1	Matériel basique	Comptage pondéré fixe des pièces
H2	Matériel dynamique	Valeur marginale décroissante des pions
H3	Avancée (H2 + centre)	H2 enrichie d'un bonus positionnel

4.3.1 H1

Cette heuristique attribue un score fixe à chaque pièce encore en jeu :

$$\text{H1}(s) = 10\times(\#\text{élèves bleus})+100\times(\#\text{maîtres bleus})-10\times(\#\text{élèves rouges})-100\times(\#\text{maîtres rouges}) \tag{1}$$

Elle est intentionnellement simple : chaque pion vaut autant, qu'il soit le dernier ou le quatrième d'une équipe. Cette heuristique correspond au niveau de difficulté **Facile**.

4.3.2 H2

H2 introduit une *valeur marginale décroissante* : plus les pions se raréfient, plus chacun prend de la valeur. Ce schéma correspond à la réalité stratégique d'Onitama, où perdre son dernier élève est une catastrophe bien plus grave que de perdre un pion parmi quatre.

$$\text{H2}(s) = f(\#\text{élèves bleus}) - f(\#\text{élèves rouges}) \tag{2}$$

où f est définie par :

$$f(n) = \begin{cases} 0 & n = 0 \\ 8 & n = 1 \\ 14 & n = 2 \\ 18 & n = 3 \\ 20 & n \geq 4 \end{cases} \tag{3}$$

Cette heuristique constitue le niveau **Moyen**.

4.3.3 H3

H3 enrichit H2 d'un bonus de positionnement qui récompense le contrôle du centre du plateau :

$$\text{H3}(s) = \text{H2}(s) + \sum_{\text{pièce } p} \text{signe}(p) \times (2 \cdot \mathbb{1}_{p \in \text{zone centrale}} + 3 \cdot \mathbb{1}_{p \in \text{case centre}}) \tag{4}$$

où $\mathbb{1}_A$ désigne la fonction indicatrice de l'ensemble A.

La *zone centrale* est définie comme $\{(x, y) \mid 1 \leq x \leq 3,\ 1 \leq y \leq 3\}$ et la *case centre* comme $(2, 2)$.

Ce bonus traduit l'importance tactique du contrôle du centre, qui accroît la mobilité des pièces et rapproche physiquement le maître des deux temples adverses. Cette heuristique constitue le niveau **Difficile**.

4.4 Profils des IA et niveaux de difficulté

Les trois niveaux de difficulté sont définis par la combinaison d'une profondeur de recherche et d'une heuristique :

Niveau	Nom	Profondeur	Heuristique
1	Facile	3	H1
2	Moyen	4	H2
3	Difficile	5	H3

Remarque : pour les expériences comparant uniquement les heuristiques (équité du test), une profondeur commune de 4 est utilisée pour les trois profils.

5 Analyse expérimentale

5.1 Temps de calcul par profondeur

TABLE 1 – Temps moyen de calcul d'un coup (en secondes) selon la profondeur et l'heuristique.

Profondeur	H1 (s)	H2 (s)	H3 (s)
1	0.0001 (±0.0000)	0.0001 (±0.0000)	0.0001 (±0.0000)
2	0.0002 (±0.0001)	0.0002 (±0.0001)	0.0005 (±0.0003)
3	0.0012 (±0.0005)	0.0014 (±0.0006)	0.0029 (±0.0013)
4	0.0069 (±0.0046)	0.0053 (±0.0048)	0.0166 (±0.0096)
5	0.0452 (±0.0369)	0.0475 (±0.0300)	0.1449 (±0.0905)
6	0.1688 (±0.0846)	0.1538 (±0.1131)	0.5727 (±0.4783)
7	1.0830 (±0.7311)	2.0322 (±2.3179)	2.8081 (±1.8955)

On observe que le temps de calcul croît de façon quasi-exponentielle avec la profondeur, conformément à la complexité théorique $O(b^{m/2})$ après élagage α-β. H3 est systématiquement plus lente que H1 et H2 en raison du parcours supplémentaire du plateau pour calculer le bonus positionnel. À profondeur 5, H3 prend environ 3 fois plus de temps que H1 ou H2 ; à profondeur 7, la différence s'accentue (2,8 s contre 1,1 s pour H1). En pratique, la profondeur 4 représente le meilleur compromis performance/temps pour H3 (16 ms en moyenne), tandis que H1 et H2 restent exploitables jusqu'à la profondeur 6.

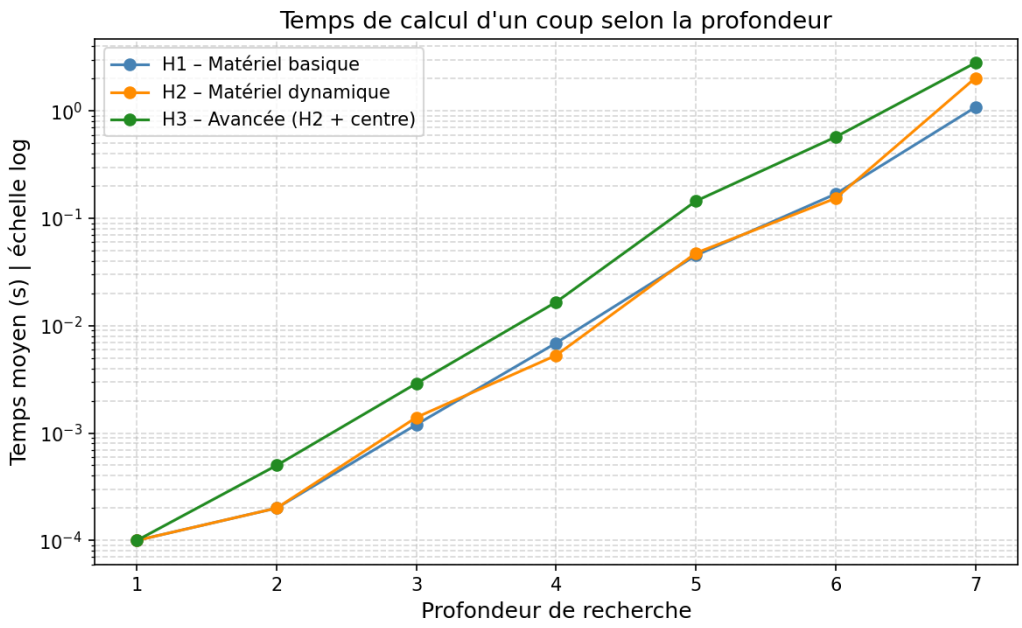

FIGURE 2 – Évolution du temps de calcul en fonction de la profondeur pour les trois heuristiques (échelle logarithmique sur l'axe des ordonnées).

5.2 Nombre de nœuds explorés

Le tableau 2 présente le nombre moyen de nœuds visités par l'algorithme Minimax avec et sans élagage α-β, mesuré avec l'heuristique H3 à profondeurs croissantes. Ces mesures ont été obtenues en ajoutant un compteur global incrémenté à chaque appel à `max_value` et `min_value`.

TABLE 2 – Nombre moyen de nœuds explorés avec et sans élagage α-β (H3).

Profondeur	Sans élagage	Avec élagage α-β	Ratio
2	175	66	2,64
3	2 441	580	4,21
4	42 591	3 109	13,70
5	852 038	24 545	34,71

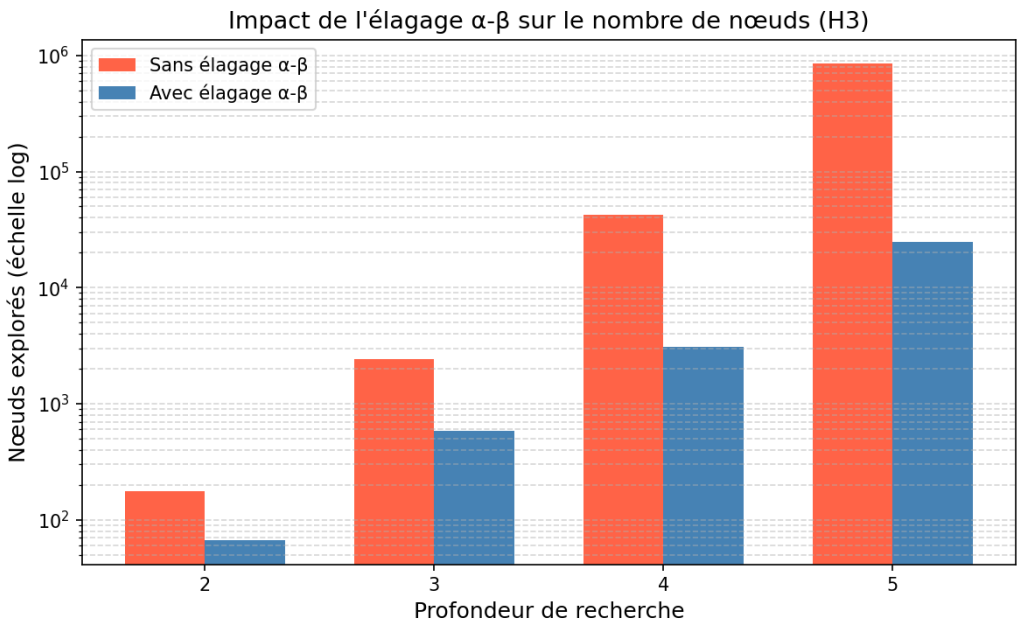

FIGURE 3 – Impact de l'élagage α-β sur le nombre de nœuds explorés selon la profondeur (H3, échelle logarithmique).

5.3 Impact de l'élagage α-β sur l'efficacité

L'élagage α-β réduit théoriquement la complexité temporelle de $O(b^m)$ à $O(b^{m/2})$ dans le meilleur cas. Dans notre contexte, le facteur de branchement moyen se situe autour d'une dizaine de coups par tour (bien qu'il puisse théoriquement atteindre la quarantaine dans de rares cas).

Les résultats du tableau 2 et de la figure 3 confirment l'efficacité croissante de l'élagage avec la profondeur : le ratio Sans/Avec passe de $\times 2{,}6$ à profondeur 2 à $\times 34{,}7$ à profondeur 5. Cette progression super-linéaire s'explique par l'effet cumulatif de l'élagage sur plusieurs niveaux de l'arbre : plus l'arbre est profond, plus la propagation des bornes α et β permet d'éliminer de branches entières.

Le mélange aléatoire des coups (`random.shuffle`) a un effet ambigu sur l'efficacité de l'élagage. Dans le meilleur cas théorique ($O(b^{m/2})$), les coups sont présentés du meilleur au pire, ce qui maximise les coupures. Notre ordre aléatoire s'écarte de cet optimum et induit une variabilité importante des ratios d'une position à l'autre ; en contrepartie, il garantit la diversité des parties et évite que l'IA explore toujours les mêmes variantes en premier.

6 Tournoi entre les intelligences artificielles

6.1 Protocole expérimental

Pour évaluer les performances relatives de nos trois niveaux de difficulté, nous avons organisé un tournoi automatisé à l'aide du script `generer_graphiques.py`. Chaque paire d'IA s'affronte dans 1000 parties. Le taux de victoire du Joueur Bleu (J1) est enregistré pour chaque croisement.

Les résultats sont présentés sous la forme d'une matrice de chaleur 3×3 : chaque cellule (i, j) indique le pourcentage de victoires de l'IA i (jouant Bleu) contre l'IA j (jouant Rouge).

6.2 Résultats

La figure 4 illustre les résultats finaux du tournoi. Les taux de victoire présentés correspondent au pourcentage de parties remportées par l'IA jouant les pièces bleues (Joueur 1) face à l'IA jouant les pièces rouges (Joueur 2).

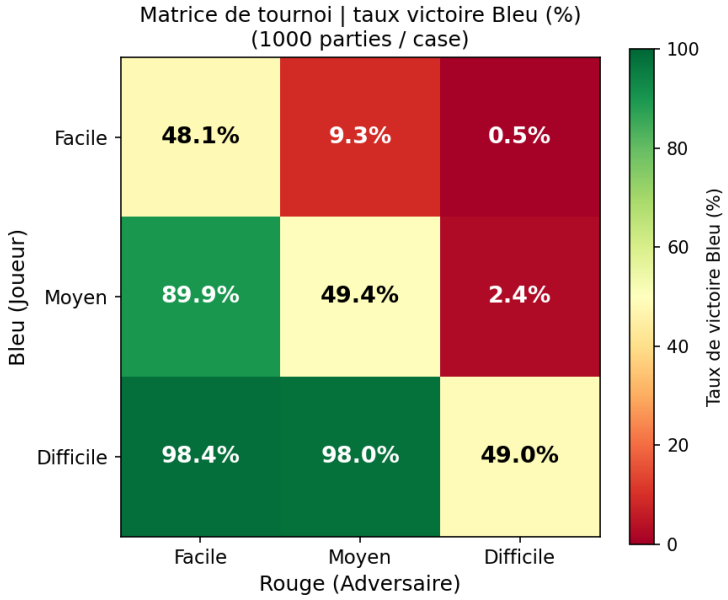

FIGURE 4 – Matrice de chaleur des taux de victoire du Joueur Bleu (%).

6.3 Évolution du taux de victoire

La figure 5 présente l'évolution du taux de victoire cumulé du joueur Bleu pour trois affrontements différents entre les IA (Facile, Moyenne et Difficile), sur 1000 parties. L'axe des abscisses est représenté en échelle logarithmique afin de mettre en évidence la convergence rapide des taux.

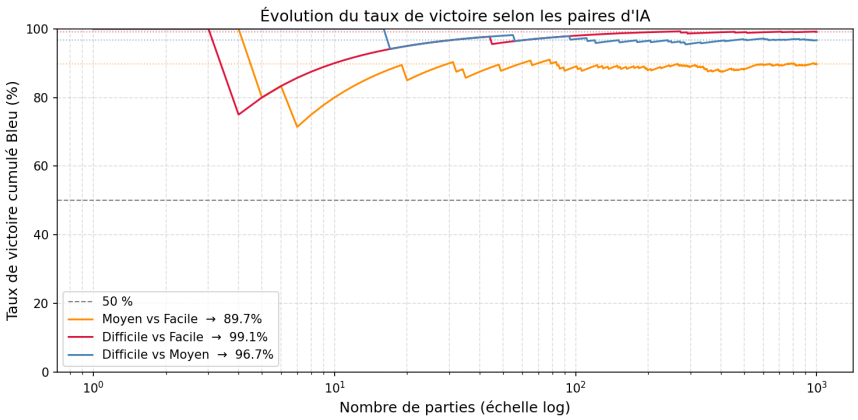

FIGURE 5 – Évolution du taux de victoire cumulé du joueur Bleu selon les paires d'IA, sur 1000 parties (axe x en échelle logarithmique). Les taux se stabilisent respectivement à 89,7%, 99,1% et 96,7%.

Dès les premières dizaines de parties, on observe quelques oscillations (particulièrement visibles sur la courbe orange) avant que les taux ne se stabilisent progressivement. À l'issue des 1000 parties, les courbes convergent vers leurs valeurs finales, démontrant une hiérarchie claire entre les niveaux :

— **Difficile vs Facile (courbe rouge) :** Le taux atteint **99,1%**, illustrant une domination quasi totale de l'IA Difficile.

— **Difficile vs Moyen (courbe bleue) :** Le taux se stabilise à **96,7%**, confirmant l'apport significatif de l'heuristique H3 et de la profondeur 5 par rapport au niveau Moyen.

— **Moyen vs Facile (courbe orange) :** Le taux converge vers **89,7%**, prouvant la nette supériorité de la profondeur 4 (H2) sur l'IA de base.

Les lignes pointillées colorées matérialisent ces taux finaux pour chaque affrontement, tandis que la ligne grise en tirets marque le seuil de 50% (égalité parfaite).

6.4 Analyse des résultats

6.4.1 Hiérarchie des niveaux

Les résultats confirment parfaitement la hiérarchie attendue entre les trois niveaux de difficulté (Facile $<$ Moyen $<$ Difficile) :

— **Moyen bat Facile** avec un taux de 89,9% (Bleu), l'avantage de H2 sur H1 et la profondeur supérieure (4 vs 3) produisent une très forte domination.

— **Difficile bat Moyen** avec un taux de 98,0% (Bleu), le bonus positionnel H3 combiné à la profondeur 5 confère un avantage quasi absolu.

— **Difficile bat Facile** avec un taux de 98,4% (Bleu), ce qui illustre l'écart massif de niveau de jeu entre les deux extrêmes.

Les matchs miroir (même IA contre elle-même) donnent des taux remarquablement équilibrés et proches de 50% : 48,1% pour Facile vs Facile, 49,4% pour Moyen vs Moyen et 49,0% pour Difficile vs Difficile. L'asymétrie potentielle liée au fait de jouer en premier ou en second semble minime, confirmant l'équité de l'affrontement lorsque les deux joueurs ont le même niveau.

6.4.2 Robustesse statistique

Avec 1000 parties par croisement, l'intervalle de confiance à 95% sur un taux de victoire p est approximativement :

$$\hat{p} \pm 1{,}96\sqrt{\frac{\hat{p}(1-\hat{p})}{1000}} \tag{5}$$

Pour $\hat{p} = 0{,}5$, cet intervalle vaut environ $\pm 3{,}1$ points de pourcentage, bien plus serré qu'avec 50 parties ($\pm 13{,}9$ pp). Les résultats des matchs miroir (compris entre 48,1% et 49,4%) s'inscrivent parfaitement dans cet intervalle de $50\% \pm 3{,}1\%$, validant l'hypothèse d'une égalité parfaite. En revanche, les résultats extrêmes ($\geq 89\%$ ou $\leq 10\%$) sont statistiquement incontestables et ne requièrent pas davantage de tirages pour conclure à une domination claire.

6.4.3 Influence des cartes aléatoires

Le tirage aléatoire des cinq cartes introduit une source de variabilité initiale dans chaque partie. Certaines combinaisons de cartes favorisent nettement un style de jeu (par exemple, des cartes offrant de grands sauts favorisent une stratégie d'encerclement), ce qui peut biaiser ponctuellement le résultat d'une manche isolée.

Cette variabilité est d'ailleurs visible dans les fortes oscillations initiales sur la courbe d'évolution du taux de victoire (figure 5). Cependant, les résultats des matchs miroirs, très proches de 50%, démontrent que les 1000 parties par croisement sont suffisantes pour échantillonner une très grande variété de tirages de cartes. La loi des grands nombres a opéré, lissant l'impact de la distribution initiale des cartes et permettant d'isoler l'impact pur des heuristiques et de la profondeur d'exploration.

7 Utilisation de l'IA générative

Dans le cadre de ce projet, nous avons eu recours à plusieurs outils d'intelligence artificielle générative, avec une répartition stricte des tâches, pour nous assister dans les phases de développement et de rédaction : les modèles **Claude Sonnet 4.6** et **Gemini 3.1 Pro** (via leurs interfaces web), ainsi que l'éditeur de code **Cursor**.

- **Type d'utilisation :** L'éditeur **Cursor** a été utilisé spécifiquement pour la documentation, facilitant la rédaction et la structuration des commentaires directement dans le code source. Le modèle **Claude** a été sollicité pour la partie programmation, principalement pour les scripts annexes (tests unitaires, génération de graphiques) et pour comprendre comment implémenter efficacement le parallélisme en Python (`concurrent.futures`). Enfin, **Gemini** a été principalement dédié au rapport : vérification mathématique des données de benchmark, relecture et mise en forme globale en LaTeX.
- **Rôle effectif :** Chaque IA a joué un rôle délimité. Cursor a permis de commenter le code de manière propre et lisible. Claude a agi comme un tuteur technique, débloquant les difficultés liées à la gestion du *multiprocessing* nécessaire pour accélérer les milliers de parties du tournoi. Gemini a endossé le rôle de relecteur et d'assistant d'édition, permettant d'aligner rigoureusement les analyses textuelles avec les sorties brutes de la console (calcul des ratios, vérification des intervalles de confiance) et de restructurer le document.
- **Analyse critique (apports et limites) :** La segmentation des tâches entre ces outils s'est révélée très pertinente. L'apport principal réside dans le gain de temps sur des tâches chronophages (commentaires, formatage des tableaux LaTeX) et dans l'aide à la compréhension de l'architecture parallèle (avec Claude). En revanche, l'utilisation de ces LLM exige une rigueur stricte dans les instructions (le *prompting*) : lors de la rédaction, il a été indispensable d'imposer à l'IA de ne formuler aucune conclusion sans recevoir au préalable les journaux d'exécution (*logs*) exacts, afin d'éviter toute hallucination de résultats ou de statistiques.

8 Difficultés rencontrées

Le développement de ce moteur de jeu et de son intelligence artificielle a soulevé plusieurs défis techniques et conceptuels, que nous avons résolus par diverses optimisations :

- **Symétrie directionnelle des cartes :** Contrairement aux échecs où les pièces ont des règles de mouvement absolues, les cartes d'Onitama sont relatives au joueur. Nous avons résolu cette difficulté en introduisant un multiplicateur de direction (`sens = 1` pour Bleu, `-1` pour Rouge) appliqué systématiquement lors de la génération des coordonnées cibles (nx, ny).
- **Implémentation du Make/Unmake :** Pour que l'algorithme Minimax explore des millions de nœuds rapidement, il était impossible de copier le plateau entier à chaque coup (`deepcopy`). La difficulté principale résidait dans l'échange de la *carte extra* : il a fallu s'assurer que la méthode `annuler_coup` réinsère la bonne carte dans

la bonne main, retire l'ancienne extra, et replace la pièce capturée éventuelle. Les tests unitaires ont été indispensables pour valider cette mécanique.

— **Temps d'exécution à haute profondeur :** Le langage Python étant interprété, le parcours de l'arbre devient très lent au-delà de la profondeur 5, particulièrement avec l'heuristique H3 qui nécessite de parcourir le plateau pour calculer les bonus de position. Pour générer nos benchmarks (comme le tournoi de 1000 parties), le temps d'exécution devenait prohibitif. Nous avons pallié ce problème en utilisant le module `concurrent.futures.ProcessPoolExecutor`, permettant de paralléliser les parties sur tous les cœurs du processeur.

— **Gestion des cas limites et parties nulles :** Deux cas limites majeurs ont été traités. D'une part, pour éviter que l'IA ne s'enferme dans des cycles infinis, une limite stricte de 1000 tours a été imposée dans la boucle de jeu principale. D'autre part, le cas très rare où un joueur n'a aucun coup légal a été simplifié : pour maintenir l'efficacité de l'arbre Minimax, le moteur passe directement le tour sans obliger l'IA à modéliser la défausse d'une carte.

— **Lissage du hasard des cartes :** Lors de nos premiers tests du tournoi sur 50 ou 250 parties, les taux de victoires montraient une forte variance liée au tirage initial des cartes. Ce n'est qu'en augmentant l'échantillon à 1000 parties par croisement que la loi des grands nombres a permis d'isoler la valeur réelle de nos heuristiques des "bons tirages" ponctuels.

9 Conclusion

Ce projet nous a permis de mettre en pratique les concepts fondamentaux de l'intelligence artificielle appliquée aux jeux de réflexion, au travers du jeu *Onitama*.

Nous avons conçu un moteur de jeu complet reposant sur un mécanisme efficace de *make/unmake*, permettant d'explorer l'arbre de jeu sans copier l'état à chaque nœud. L'algorithme Minimax avec élagage α-β a été intégré dès la racine, réduisant substantiellement le nombre de nœuds évalués.

Trois heuristiques de complexité croissante ont été développées : H1 (comptage matériel fixe), H2 (valeur marginale décroissante) et H3 (H2 augmentée d'un bonus positionnel centré). Les tournois automatisés ont confirmé la hiérarchie attendue entre les niveaux de difficulté : Facile $<$ Moyen $<$ Difficile, avec des écarts de performance très marqués (allant de près de 90% à plus de 98% de victoires d'un niveau sur l'inférieur). L'élagage α-β s'est révélé particulièrement efficace, réduisant de près d'un facteur 35 ($\times 34{,}7$) le nombre de nœuds explorés à profondeur 5. H3 offre les meilleures performances stratégiques au prix d'un surcoût de calcul d'un facteur 3 environ par rapport à H1 et H2, coût qui reste acceptable jusqu'à la profondeur 5 (environ 145 ms par coup).

Plusieurs pistes d'amélioration sont envisageables pour repousser les limites de profondeur. Pour l'approche Minimax, l'ordonnancement des coups avant l'élagage (tri par score heuristique), l'utilisation de tables de transposition pour la mémoïsation des états [4], ou encore l'exploration itérative en profondeur (IDA*) permettraient d'optimiser drastiquement les coupures α-β [5]. Alternativement, l'implémentation d'une approche probabiliste comme la recherche arborescente Monte-Carlo (MCTS) [6], particulièrement efficace sur des jeux à fort facteur de branchement, constituerait un prolongement naturel à ce projet.

Références

[1] Arcane Wonders. *Onitama | Livret de règles officiel*.
Disponible sur : https://www.arcanewonders.com/wp-content/uploads/2021/05/Onitama-Rulebook.pdf

[2] Wikipédia. *Onitama* (Article en anglais).
Disponible sur : https://en.wikipedia.org/wiki/Onitama

[3] Jack Adamson. *Onitama* (Dépôt GitHub).
Disponible sur : https://github.com/jackadamson/onitama

[4] C#ESS Enginer. *Episode_08_11 : La table de transposition* (Vidéo en ligne). YouTube, 2021.
Disponible sur : https://www.youtube.com/watch?v=gBOIFDDUjcE

[5] Mahesh Huddar. *Iterative Deepening A Star algorithm IDA Star | IDA* – Artificial intelligence by Mahesh Huddar* (Vidéo en ligne). YouTube, 2024.
Disponible sur : https://www.youtube.com/watch?v=BUHc8p5Mpdo

[6] John Levine. *Monte Carlo Tree Search* (Vidéo en ligne). YouTube, 2017.
Disponible sur : https://www.youtube.com/watch?v=UXW2yZndl7U

A Code source principal

A.1 Évaluation heuristique H2 - Matériel dynamique

```python
def evaluation_materiel_dynamique(self):
    def points_par_pions(compte):
        if compte == 0: return 0
        if compte == 1: return 8
        if compte == 2: return 14
        if compte == 3: return 18
        if compte >= 4: return 20

    pions_bleus = pions_rouges = 0
    maitre_bleu = maitre_rouge = False

    for y in range(5):
        for x in range(5):
            p = self.plateau[y][x]
            if p == 1:  pions_bleus  += 1
            elif p == 2:  maitre_bleu   = True
            elif p == -1: pions_rouges += 1
            elif p == -2: maitre_rouge  = True

    if not maitre_bleu:  return -10000
    if not maitre_rouge: return  10000
    return points_par_pions(pions_bleus) - points_par_pions(
        pions_rouges)
```

Listing 4 – Heuristique H2 : valeur marginale décroissante des pions.

A.2 Script de génération des graphiques

Le script `generer_graphiques.py` automatise les benchmarks et les tournois, puis produit quatre graphiques :

- `temps_calcul.png` : temps moyen par coup selon la profondeur ;
- `noeuds_explores.png` : nœuds explorés avec et sans élagage α-β ;
- `matrice_resultats.png` : matrice de chaleur 3×3 des taux de victoire ;
- `evolution_victoires.png` : courbe du taux de victoire cumulé selon le nombre de parties(axe x en échelle logarithmique).

Pour l'exécuter : `python3 generer_graphiques.py`

B Dictionnaire des 16 cartes

TABLE 3 – Les 16 cartes d'Onitama avec leurs déplacements $(\Delta x, \Delta y)$ et leur couleur de départ.

Carte	**Couleur**	**Déplacements** $(\Delta x, \Delta y)$
Tiger	Bleu	$(0, 2)$, $(0, -1)$
Dragon	Rouge	$(-2, 1)$, $(-1, -1)$, $(1, -1)$, $(2, 1)$
Frog	Rouge	$(-2, 0)$, $(-1, 1)$, $(1, -1)$
Rabbit	Bleu	$(-1, -1)$, $(1, 1)$, $(2, 0)$
Crab	Bleu	$(-2, 0)$, $(0, 1)$, $(2, 0)$
Elephant	Rouge	$(-1, 1)$, $(-1, 0)$, $(1, 0)$, $(1, 1)$
Goose	Bleu	$(-1, 1)$, $(-1, 0)$, $(1, 0)$, $(1, -1)$
Rooster	Rouge	$(-1, -1)$, $(-1, 0)$, $(1, 0)$, $(1, 1)$
Monkey	Bleu	$(-1, 1)$, $(-1, -1)$, $(1, 1)$, $(1, -1)$
Mantis	Rouge	$(-1, 1)$, $(0, -1)$, $(1, 1)$
Horse	Rouge	$(-1, 0)$, $(0, 1)$, $(0, -1)$
Ox	Bleu	$(0, 1)$, $(0, -1)$, $(1, 0)$
Crane	Bleu	$(0, 1)$, $(-1, -1)$, $(1, -1)$
Boar	Rouge	$(0, 1)$, $(-1, 0)$, $(1, 0)$
Eel	Bleu	$(-1, 1)$, $(-1, -1)$, $(1, 0)$
Cobra	Rouge	$(-1, 0)$, $(1, 1)$, $(1, -1)$

Note : les déplacements sont relatifs à la direction du joueur. Le Joueur 2 (Rouge) joue avec sens $= -1$, ce qui inverse les coordonnées y par rapport au Joueur 1. La couleur de la carte extra détermine quel joueur commence la partie.